CS331: Data Communications and Computer Networks

Wireshark Lab

Packet Capture and Display Filtering

SHOUG FAWAZ ABDULLAH ALOMRAN

Instructor: Ms.Nada ALALYANI

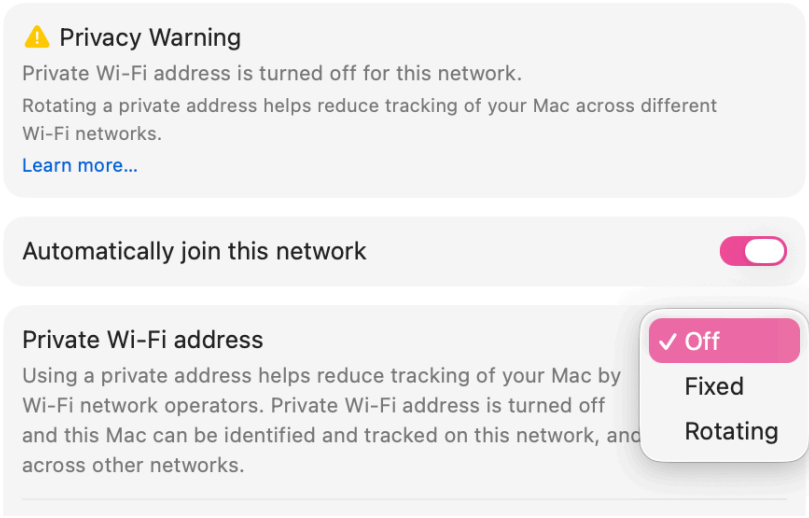
Part 1: Lab Setup **3**
Step 1: Finding the IP and MAC address 3
Step 2: Starting Wireshark 3
Steps 3–4: Capturing packets 4
Step 5-12: Filtering 4
Part 2: Packet Filtering Practice **5**
Q1. IP address of Google.com.sa: 142.250.203.3 5
Q2. Time for the first Echo packet: 121.913 ms 6
Q3. Active interface: 6
Q4. Filter: icmp and ip.src == 172.217.17.35 7
Q5. Filter: icmp 7
Q6. ICMP packets filtered: 7
Q7. Packets with my IP as source: 7
Q8. Packets with Google's IP as source: 7
Q9. Frame source and destination addresses: 9
Q11. Type field value: 9
Q12. Source and destination IP addresses: 9
Q13. Time for the Echo response: 9

Part 1: Lab Setup

Note: This lab was performed on macOS (MacBook Air M2). macOS equivalents were used for the Windows commands: ifconfig en0 instead of ipconfig /all, and ping -c 4 instead of **ping -4.**

Step 1: Finding the IP and MAC address

The private Wi-Fi Address was turned off so the real hardware MAC address is shown.

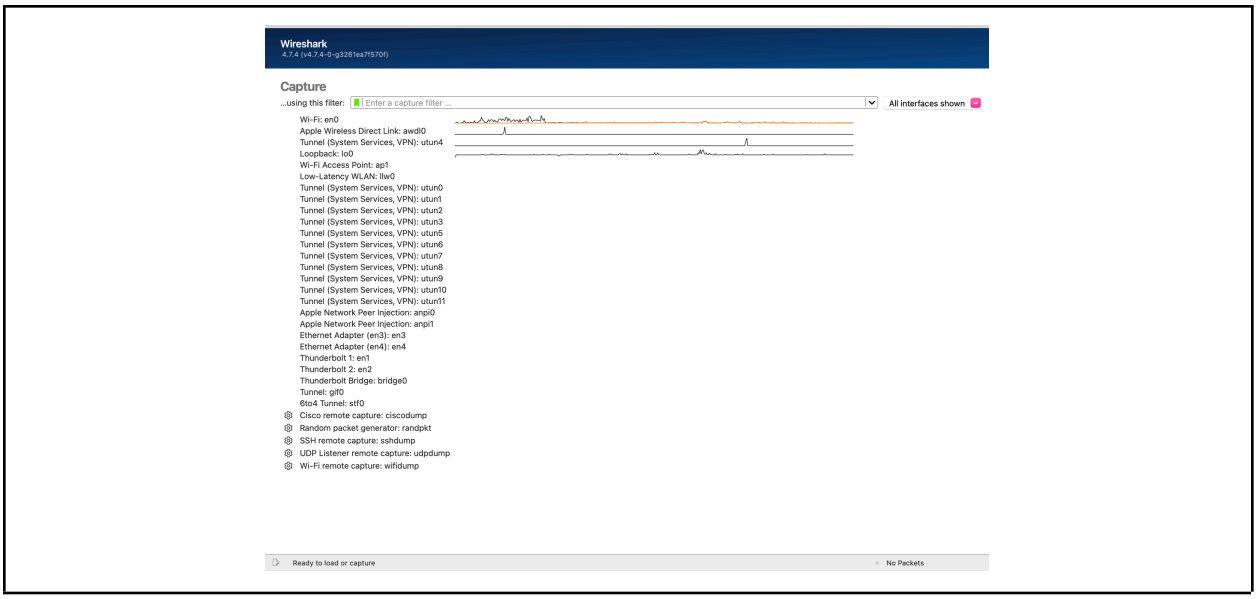


Command used: ifconfig en0

```
shougalomran@Mac ~ % ifconfig en0
en0: flags=8863<UP,BROADCAST,SMART,RUNNING,SIMPLEX,MULTICAST> mtu 1500
        options=6460<TSO4,TSO6,CHANNEL_IO,PARTIAL_CSUM,ZEROINVERT_CSUM>
        ether a8:8f:d9:xx:xx:xx
        inet6 fe80::188b:dacc:bfea:b9c6%en0 prefixlen 64 secured scopeid 0xb
        inet 192.168.1.229 netmask 0xffffff00 broadcast 192.168.1.255
        nd6 options=201<PERFORMNUD,DAD>
        media: autoselect
        status: active
shougalomran@Mac ~ %
```

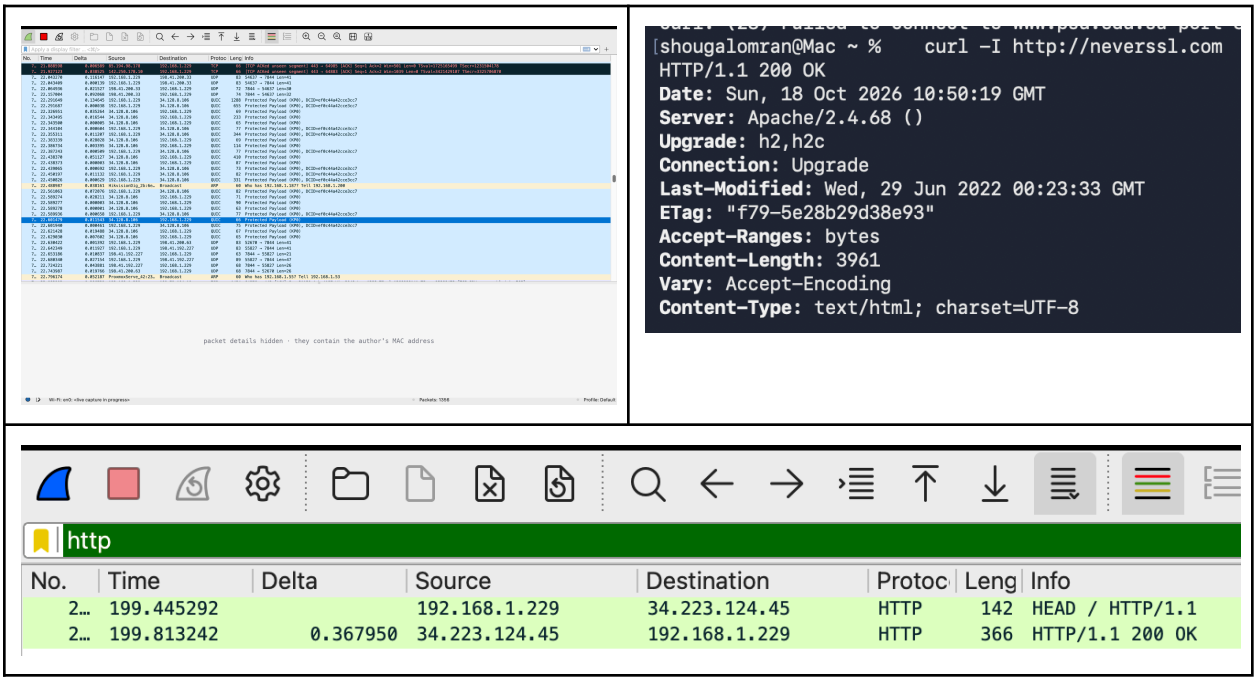
Step 2: Starting Wireshark

The active interface Wi-Fi: en0 was selected for capturing.



Steps 3–4: Capturing packets

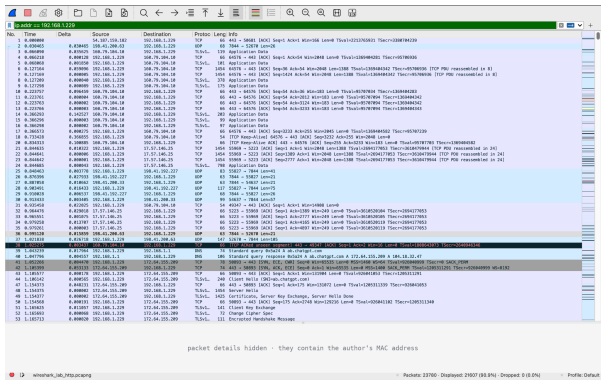
While capturing, www.psu.edu.sa was opened in the browser and an HTTP request was sent to neverssl.com, then the capture was stopped.



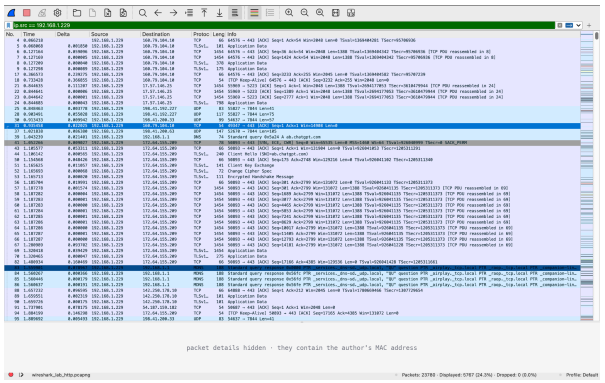
Step 5-12: Filtering

Note: www.psu.edu.sa only accepts HTTPS (port 80 is closed), so its traffic appears as encrypted TLS rather than HTTP. A plain HTTP request was sent to http://neverssl.com to

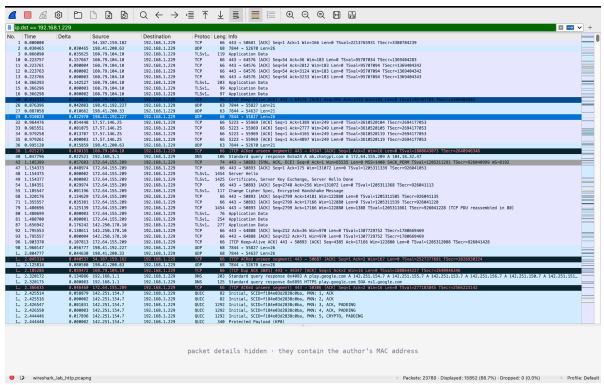
demonstrate the http filter. The result shows the HEAD request from my machine and the HTTP/1.1 200 OK response.



All packets to or from my machine.



Only packets sent by my machine.



Only packets received by my machine.



All DNS and HTTP packets.



Only HTTP packets received by my machine.

Part 2: Packet Filtering Practice

Q1. IP address of Google.com.sa: 142.250.203.3

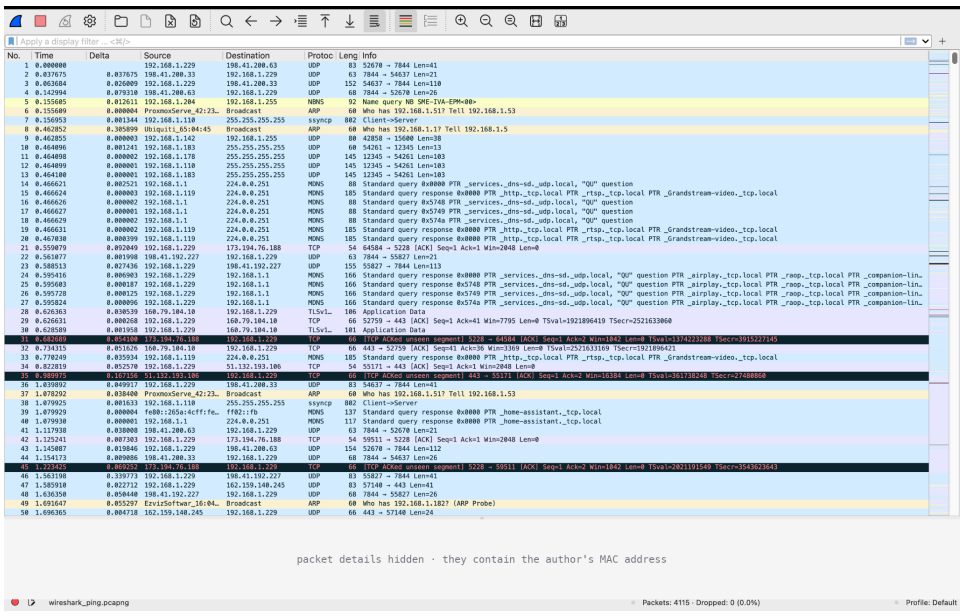
```
[shougalomran@Mac ~ % nslookup google.com.sa 8.8.8.8
Server:         8.8.8.8
Address:        8.8.8.8#53

Non-authoritative answer:
Name:   google.com.sa
Address: 142.250.203.3
```

```
[shougalomran@Mac ~ % ping -c 4 google.com.sa
PING google.com.sa (172.217.17.35): 56 data bytes
64 bytes from 172.217.17.35: icmp_seq=0 ttl=114 time=121.913 ms
64 bytes from 172.217.17.35: icmp_seq=1 ttl=114 time=91.989 ms
64 bytes from 172.217.17.35: icmp_seq=2 ttl=114 time=92.667 ms
64 bytes from 172.217.17.35: icmp_seq=3 ttl=114 time=106.484 ms

--- google.com.sa ping statistics ---
4 packets transmitted, 4 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 91.989/103.263/121.913/12.223 ms
shougalomran@Mac ~ %
```

Note: nslookup via 8.8.8.8 returned 142.250.203.3, while the ping resolved google.com.sa through the local DNS server (192.168.1.1) to 172.217.17.35. Google uses many server IPs, so different DNS resolvers can return different addresses. The captured ICMP packets use 172.217.17.35.

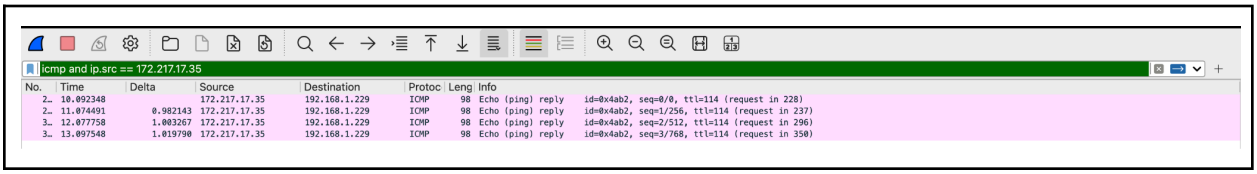


Q2. Time for the first Echo packet: 121.913 ms

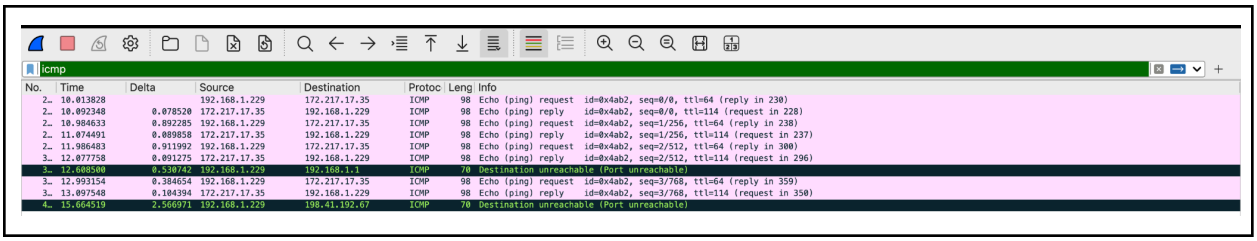
Q3. Active interface:

MAC address	a8:8f:d9:xx:xx:xx
IP address	192.168.1.229

Q4. Filter: icmp and ip.src == 172.217.17.35



Q5. Filter: icmp



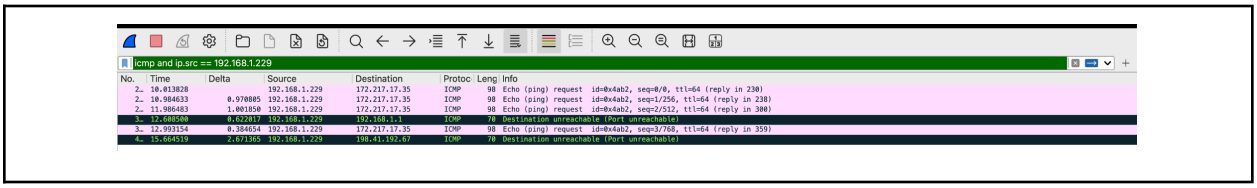
Q6. ICMP packets filtered:

10 (8 ping packets: 4 Echo requests + 4 Echo replies, plus 2 Destination Unreachable messages from other background traffic)

Q7. Packets with my IP as source:

6 (4 Echo requests + 2 Destination Unreachable)

Filter used: `icmp and ip.src == 192.168.1.229`



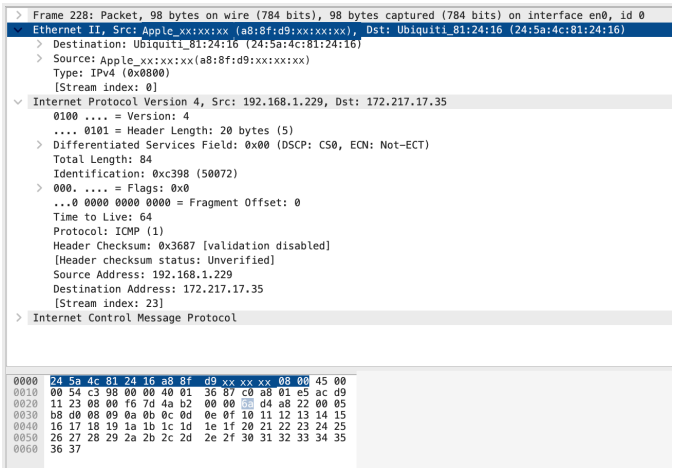
Q8. Packets with Google's IP as source:

4 (the Echo replies)

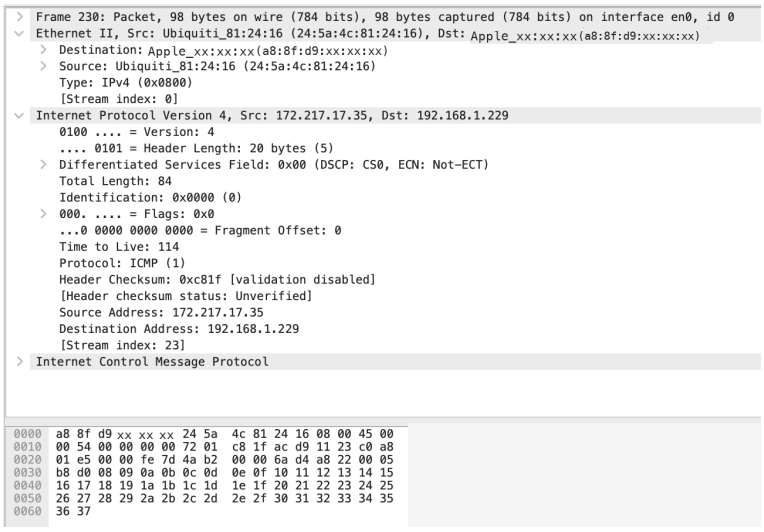
Filter used: `icmp and ip.src == 172.217.17.35`

Using the first Echo request (frame 228):





Using the first Echo request (frame 230):



Q9. Frame source and destination addresses:

Source MAC: a8:8f:d9:xx:xx:xx (my Mac)

Destination MAC: 24:58:4c:81:24:16 (Ubiquiti router)

Q10. Is the destination MAC the address of Google.com.sa?

No. It is the MAC address of the default gateway (my router, 192.168.1.1). MAC addresses are only used for delivery within the local network, hop by hop. Since Google is on a different network, the frame is sent to the router, which forwards the packet toward Google. The IP addresses stay the same end to end, but the MAC addresses change at every hop.

Q11. Type field value:

0x0800, which means the Ethernet frame carries an IPv4 packet.

Q12. Source and destination IP addresses:

Source: 192.168.1.229, Destination: 172.217.17.35

Q13. Time for the Echo response:

Echo request (frame 228): 10.013828 s

Echo reply (frame 230): 10.092348 s

10.092348 − 10.013828 = 0.078520 s = 78.52 ms

It is lower than the 121.913 ms in Q2. Wireshark timestamps packets at the network interface, so it measures only the network round-trip time. The ping command measures in the application, so its time also includes operating-system processing delay, which is usually largest for the first packet.